

Uso mucho Claude Desktop. Desktop Commander me consume los tokens muy rápido. Tengo un repo en GitHub linkado como documentación de proyecto, y lo que suelo hacer es que lea el código del repo y aplique los cambios en la carpeta, luego hago commit con GitHub Desktop y así stá todo sincronizado, pero los tokens me duran muy poco. Cuál sería una mejor estrategia?

Voy a separar el problema en dos partes: reducir contexto inútil y cambiar el flujo de edición para que el modelo toque solo los archivos necesarios. También voy a contrastar el estado actual de las herramientas relevantes para no basarme en supuestos desactualizados.

Pensó durante 2m 56s >

Sí: **la mejor estrategia es dejar de usar Claude Desktop + Desktop Commander como “agente principal de código”**. Úsalo para pensar, revisar o planificar, pero haz los cambios con una herramienta diseñada para trabajar sobre repositorios locales con gestión de contexto.

Mi recomendación práctica:

Estrategia recomendada

Usa este flujo:

GitHub Desktop / git local → Claude Code o Aider → tests → commit con GitHub Desktop

Claude Desktop con Desktop Commander funciona, pero tiende a gastar muchos tokens porque cada lectura de archivo, listado de carpetas, búsqueda y salida de terminal acaba entrando en el contexto de la conversación. Incluso Anthropic advierte en sus buenas prácticas de Claude Code que el contexto se llena con mensajes, archivos leídos y salidas de comandos, y que el rendimiento empeora al llenarse. 🌸

Para código, **Claude Code** está pensado justo para esto: leer el codebase, editar archivos, ejecutar comandos y trabajar dentro de tus herramientas de desarrollo. 🌸 Desktop Commander, en cambio, es más un MCP generalista para controlar archivos, procesos y terminal desde Claude Desktop. Tiene muchísimas capacidades, pero eso mismo puede volverlo muy ruidoso en tokens si lo usas para explorar repos enteros. 🐙

Lo que cambiaría en tu caso

No linkaría el repo entero como “documentación de proyecto” salvo que sea pequeño. En Claude Projects, la knowledge base sirve para aportar contexto persistente al chat. 🌟 Pero un repo completo como documentación suele ser mala materia prima: hay demasiados archivos irrelevantes, código generado, tests, lockfiles, configs, dependencias, etc.

En vez de eso, pondría en el proyecto solo:

```
docs/ARCHITECTURE.md
docs/CONVENTIONS.md
docs/API_OVERVIEW.md
docs/COMMON_TASKS.md
README.md
CLAUDE.md o AI_INSTRUCTIONS.md
```



Y dejaría el código real en local para que el agente lo inspeccione bajo demanda.

Flujo ideal

1. Mantén el repo local como fuente de verdad

Clona el repo normalmente y sigue usando GitHub Desktop para commits, ramas y sync.

```
</> Bash
```



```
git checkout -b feature/nombre-cambio
```

2. Usa Claude Code desde la raíz del repo

En lugar de pedirle a Claude Desktop que “lea el repo”, abre el terminal dentro del proyecto y usa Claude Code.

```
</> Bash
```



```
cd tu-repo
claude
```

Claude Code tiene memoria de proyecto mediante archivos como `CLAUDE.md`, que se cargan al inicio de la sesión junto con otras memorias del proyecto. Anthropic recomienda que esas instrucciones sean específicas y concisas. 🌟

3. Crea un `CLAUDE.md` pequeño

Ejemplo:

<> Markdown



```
# Instrucciones del proyecto

## Stack
- Next.js + TypeScript
- Prisma
- Tailwind
- PostgreSQL

## Comandos
- Instalar: pnpm install
- Dev: pnpm dev
- Tests: pnpm test
- Lint: pnpm lint
- Typecheck: pnpm typecheck

## Reglas
- No leer el repositorio completo salvo que lo pida explícitamente.
- Usar rg para localizar símbolos antes de abrir archivos.
- Leer solo los archivos necesarios.
- No reescribir archivos completos si basta con un patch pequeño.
- Ejecutar lint/typecheck después de cambios relevantes.
- Mostrar siempre resumen y git diff final.
```

Esto vale oro: reduces exploración inútil y evitas que el agente “se pase de listo” leyendo medio proyecto.

4. Dale tareas pequeñas, no “lee todo y cambia”

Malo:

Lee todo el repo, entiende cómo funciona y añade X.



Bueno:

Quiero añadir validación de email al formulario de registro.



```
Primero localiza los componentes y servicios relevantes usando rg.
No leas más de 8 archivos salvo que sea necesario.
Haz un plan breve antes de editar.
Aplica el cambio mínimo.
Ejecuta typecheck/test si procede.
Al final dame resumen y diff.
```

Este cambio de prompt reduce muchísimo consumo.

5. Usa Claude Desktop solo para diseño o revisión

Por ejemplo:

```
Te paso este diff. Revisalo buscando bugs, edge cases y problemas de arquitectura.
```



O:

```
Tengo esta feature. Diseña un plan de implementación en pasos pequeños para dárselo
```



No usaría Claude Desktop + Desktop Commander para hacer navegación profunda de repo salvo tareas puntuales.

Alternativa muy buena: Aider

Si quieres algo más ligero y centrado en commits/diffs, **Aider** también encaja bien. Su ventaja es que usa un “repo map”: crea un mapa del repositorio y selecciona las partes relevantes según el presupuesto de tokens, en vez de meter todo el repo en contexto. 

Flujo:

```
</> Bash
```



```
cd tu-repo  
aider
```

Y le pides cambios concretos. Aider funciona muy bien cuando quieres que el asistente modifique archivos y tú revises el diff.

Mi recomendación concreta

Para tu caso usaría esto:

Opción principal: Claude Code local.

Opción secundaria si quieres más control de diffs: Aider.

Mantendría GitHub Desktop igual para revisar, commitear y sincronizar.

Dejaría Claude Desktop + Desktop Commander solo para tareas puntuales, no para edición continua del repo.

El mayor ahorro vendrá de estas tres reglas:

1. **No subir ni linkar el repo entero como documentación.**
2. **No pedir “lee el código del repo”;** pedir “localiza lo relevante con `rg`”.
3. **Trabajar por cambios pequeños, con sesiones limpias y contexto mínimo.**

Un prompt base que puedes reutilizar:

Trabaja con mínimo contexto.



Objetivo: [describe el cambio]

Reglas:

- No leas el repositorio completo.
- Usa rg/find para localizar archivos relevantes.
- Lee solo los archivos necesarios.
- Haz un plan breve antes de editar.
- Aplica el cambio mínimo.
- No pegues archivos completos en la respuesta.
- Ejecuta tests/typecheck/lint si aplica.
- Al final muestra resumen, archivos tocados y riesgos.

Ese cambio de workflow debería reducir bastante el consumo sin perder la comodidad de tener todo sincronizado con GitHub Desktop.



README.md
Archivo



CONTEXT.md
Archivo

Esta es la documentación del proyecto. Dime todo lo que debería montar para empezar a ahorrar tokens

Pensó durante 45s >

Sí. Con esta documentación, yo montaría un **sistema de contexto mínimo para agentes**, no metería el repo entero como documentación del proyecto.

Tu proyecto ya tiene una estructura clara: backend en `01_Core`, herramienta HTML en `02_Herramienta`, diagnósticos/tests en `03_Diagnostico`, docs en `05_Docs`, backups y obsoletos separados. Eso es perfecto para ahorrar tokens si obligas al agente a trabajar por zonas, no por repo completo.  CONTEXT

Qué montaría

1. Claude Code o Aider como herramienta principal de edición

Mantén **GitHub Desktop** para revisar diffs, commits y sync.

Pero para editar código, cambia este flujo:

Claude Desktop + Desktop Commander lee carpetas → edita archivos → mucho contexto



por este:

Claude Code/Aider en la raíz del repo → lee solo archivos relevantes → edita → test



Claude Code está pensado para leer archivos, ejecutar comandos y editar dentro del repo. Además, Claude Code carga instrucciones persistentes desde `CLAUDE.md`, y puedes controlar qué contexto inicial recibe cada sesión. 🌸🌸

2. Un `CLAUDE.md` corto en la raíz del repo

Archivo:

C:\Users\Pc\Desktop\PROYECTOS\WALLAPOP\CLAUDE.md



Contenido recomendado:

</> Markdown



```
# Wallapop Alerts - instrucciones para agentes

## Regla principal
Trabaja con mínimo contexto. No leas el repositorio completo. Usa búsquedas dirigidas (`r

## Estructura del repo
- `01_Core/`: backend Python. Archivos principales: `main.py`, `scraper.py`, `classifier.
- `02_Herramienta/`: configurador HTML autocontenido. Archivo actual: `wallapop_config_v
- `03_Diagnostico/`: scripts de diagnóstico y tests sin red.
- `05_Docs/`: documentación larga. Consultar solo si la tarea lo requiere.
- `06_Backups/`, `99_Obsoletos/`, `04_Logs/`: no modificar salvo petición explícita.

## Reglas de trabajo
- Antes de editar, explica qué archivos crees que tocarás.
- No abras `alerts.db`, backups, logs ni obsoletos salvo petición explícita.
- No pegues archivos completos en la conversación.
- No reescribas YAMLS comentados con `yaml.dump` si se pueden editar de forma localizada.
- No cambies secretos ni credenciales.
- Si tocas backend, ejecuta los tests/diagnósticos relevantes.
- Si tocas GitHub Actions, recuerda que `alerts.db` y `configs/` son estado versionado.
- Al final muestra: resumen, archivos modificados, comandos ejecutados, riesgos y siguen

## Comandos útiles

Backend:
```powershell
cd C:\Users\Pc\Desktop\PROYECTOS\WALLAPOP\01_Core
python3 main.py --once --force
python3 config_inbox.py --dry-run
python3 manage.py list
```

Diagnóstico:

<> PowerShell



```
cd C:\Users\Pc\Desktop\PROYECTOS\WALLAPOP\03_Diagnostico
python3 test_delivery.py
python3 test_cascade.py
python3 test_llm_cloud.py
python3 test_new_providers.py
python3 test_price_drops.py
python3 test_add_alerts.py
python3 diagnostico.py
```

## Flujo de implementación

1. Localiza con `rg`.
2. Lee solo los archivos mínimos.
3. Haz un plan breve.
4. Edita.
5. Ejecuta validación.
6. Revisa `git diff`.
7. Resume.



```
Esto solo ya te va a ahorrar mucho porque convierte al agente en "buscador quirúrgi

3. Un `CLAUDE.local.md` privado

Archivo:

```text
C:\Users\Pc\Desktop\PROYECTOS\WALLAPOP\CLAUDE.local.md
```

Y añádelo al `.gitignore`.

Contenido:

<> Markdown



```
# Preferencias personales de Darío

- Responder en español.
- No hacer cambios grandes sin plan previo.
- Priorizar soluciones simples y mantenibles.
- Usar GitHub Desktop para commits; el agente no debe commitear salvo petición explícita.
- No ejecutar `main.py` en local si el workflow de GitHub Actions está activo, salvo que
- Cuando haya duda, hacer cambios pequeños y verificables.
```

Claude Code permite separar instrucciones versionadas (`CLAUDE.md`) de preferencias locales no versionadas (`CLAUDE.local.md`). 🌟

4. Un resumen ultracorto para sustituir README + CONTEXT en Claude Desktop

Tus `README.md` y `CONTEXT.md` son muy útiles, pero son demasiado largos para usarlos como contexto base en cada conversación. El README explica instalación, uso, Ollama, configs por correo, sueño nocturno, usuarios, diagnóstico y lógica de clasificación.

 README El `CONTEXT.md` además incluye estructura interna, decisiones históricas, despliegue, workflow, HTML, build pipeline y diseño futuro.  CONTEXT

Crea este archivo:

05_Docs/AI_BRIEF.md



Contenido:

</> Markdown



```
# AI_BRIEF — Wallapop Alerts
```

```
Proyecto Python + HTML para alertas de juegos de mesa en Wallapop.
```

```
## Partes
```

- ``01_Core/``: backend Python.
- ``02_Herramienta/``: HTML configurador actual ``wallapop_config_v18.html``.
- ``03_Diagnostico/``: scripts de diagnóstico y tests.
- ``05_Docs/``: documentación larga.
- GitHub Actions ejecuta el bot cada hora y commitea ``alerts.db`` + ``configs/``.

```
## Backend principal
```

- ``main.py``: orquesta ciclos, sueño nocturno, multi-config, novedades, bajas, bajadas de
- ``scraper.py``: API Wallapop + paginación.
- ``classifier.py``: reglas + LLM cascade + circuit breaker.
- ``database.py``: SQLite ``alerts.db``.
- ``notifier.py``: emails Gmail.
- ``config_inbox.py``: lee configs por correo.
- ``manage.py``: alta/baja/listado de usuarios.

```
## Reglas importantes
```

- Ante la duda, dejar pasar anuncios.
- El título decide relevancia; el LLM clasifica base/expansión/componentes/lote/no-juego.
- No editar configs comentados con dumps genéricos.
- No ejecutar localmente ``main.py`` con Actions activo salvo intención clara.
- No tocar secretos.

```
- No modificar `06_Backups`, `99_Obsoletos` ni `04_Logs` salvo petición explícita.

## Validaciones
- Backend: tests de `03_Diagnostico/`.
- Config inbox: `python3 config_inbox.py --dry-run`.
- Ciclo manual: `python3 main.py --once --force`.
- Usuarios: `python3 manage.py list`.

## Documentación larga
- `README.md`: uso e instalación.
- `CONTEXT.md`: arquitectura, decisiones, deploy, HTML, historial.
- `05_Docs/DEPLOY_GITHUB_ACTIONS.md`: workflow.
- `05_Docs/ALTA_USUARIOS.md`: usuarios.
```

En Claude Desktop Project Knowledge, pondría solo:

```
AI_BRIEF.md
CLAUDE.md
quizá README.md solo si necesitas soporte funcional
```

No pondría `CONTEXT.md` entero como contexto permanente. Lo dejaría para consultas puntuales.

5. Un archivo de “rutas prohibidas / caras”

Crea:

```
05_Docs/AI_NO_READ_BY_DEFAULT.md
```

Contenido:

↔ Markdown

```
# Archivos/carpetas que el agente no debe leer por defecto
```

```
No leer salvo petición explícita:
```

```
- `alerts.db`
- `04_Logs/`
- `06_Backups/`
- `99_Obsoletos/`
- versiones antiguas del HTML: `wallapop_config_v15.html`, `wallapop_config_v17.html`
- respuestas crudas de Wallapop
- zips históricos
- documentación larga completa si basta con `AI_BRIEF.md`
```

```
Motivo: ruido, coste de tokens y riesgo de confundir estado histórico con estado actual.
```

Y en `CLAUDE.md` añade:

</> Markdown



Consultar ``05_Docs/AI_NO_READ_BY_DEFAULT.md`` antes de explorar carpetas grandes.

6. Un archivo de workflows por tipo de tarea

Crea:

`05_Docs/AI_WORKFLOWS.md`



Contenido:

</> Markdown



```
# Workflows para agentes

## Cambio en backend Python
1. Leer `AI_BRIEF.md`.
2. Buscar símbolos con `rg`.
3. Abrir solo archivos de `01_Core/` relacionados.
4. Si afecta clasificación: revisar `classifier.py`, `main.py`, tests de `03_Diagnostico/`
5. Ejecutar tests relevantes.
6. Mostrar diff.

## Cambio en configurador HTML
1. Si es cambio menor, editar `02_Herramienta/wallapop_config_v18.html`.
2. Si es cambio estructural, revisar fuente React si está disponible.
3. Respetar que v18 es la versión actual.
4. No tocar v15/v17 salvo referencia histórica.

## Cambio en usuarios/configs
1. Preferir `manage.py`.
2. No editar `bot_settings.yaml` a ciegas.
3. No sobrescribir YAMLS comentados con serialización completa.
4. Verificar con `python3 manage.py list`.

## Cambio en GitHub Actions
1. Leer sección deploy en `CONTEXT.md`.
2. Recordar que el workflow commitea `alerts.db` y `configs/`.
3. Mantener `git pull --rebase --autostash` antes de `git add`.
4. No romper el gate de sueño nocturno.

## Cambio en clasificador
1. Revisar `classifier.py`.
2. Revisar tests `test_cascade.py`, `test_llm_cloud.py`, `test_new_providers.py`.
3. Mantener fail-fast ante 429.
```

4. Mantener fallback `rules`.
5. Ante duda, preferir false positive a anuncio perdido.

Esto evita que tengas que reexplicar cada vez cómo tocar cada parte.

7. Un prompt fijo para usar con Claude Code

Guárdalo como snippet:

Trabaja con mínimo contexto.

Objetivo:

[describe aquí el cambio]

Reglas:

- No leas el repo completo.
- Usa `rg`/`git grep` para localizar archivos relevantes.
- Lee como máximo 8 archivos antes de proponer plan.
- No abras `04\_Logs`, `06\_Backups`, `99\_Obsoletos`, `alerts.db` ni versiones antiguas.
- Antes de editar, dime qué archivos tocarás.
- Haz el cambio mínimo.
- Ejecuta tests/diagnósticos relevantes.
- No hagas commit.
- Al final dame resumen, archivos tocados, comandos ejecutados, riesgos y diff conciso.

Para tareas pequeñas:

Localiza dónde se implementa [X], lee solo los archivos necesarios y dime el plan.

Luego:

Aplica el plan con el cambio mínimo y valida con los tests relevantes. No hagas commit.

Separar **localización** → **plan** → **edición** suele ahorrar más tokens que pedir “arréglalo” directamente.

8. Un flujo de ramas barato

Para cada cambio:

PowerShell

```
git checkout main
git pull
```

```
git checkout -b fix/nombre-corto
claude
```

Después:

```
</> PowerShell
```

```
git diff
```

Revisas en GitHub Desktop y commiteas tú.

No dejes que el agente haga commits salvo tareas mecánicas. Tu documentación indica que GitHub Actions usa un patrón de commit-back para persistir `alerts.db` y `configs/`, así que conviene mantener muy claro cuándo un commit es humano y cuándo es del workflow. [CONTEXT](#)

9. Un setup concreto para Desktop Commander

Si sigues usando Claude Desktop, úsalo así:

Permitido:

- leer 1-3 archivos concretos
- revisar un diff pegado
- ayudar a diseñar una solución
- generar un prompt para Claude Code

Evitar:

- "lee todo el repo"
- "explora la carpeta"
- "búscame todo lo relacionado"
- "aplica cambios en muchos archivos"
- "revisa todos los docs"

Desktop Commander lo usaría como bisturí, no como IDE agent.

10. Una limpieza de secretos antes de usar agentes con más frecuencia

En tus documentos aparece la App Password de Gmail en texto plano y además se indica que está en configs, `bot_settings.yaml` y hardcodeada en el formulario. [CONTEXT](#)

Yo haría esto antes de expandir el uso de agentes:

1. Revocar esa App Password en Google.
2. Crear una nueva.
3. Pasarla a GitHub Secrets.
4. Evitar que el HTML final contenga la app password si no es estrictamente necesaria.
5. Añadir al `CLAUDE.md`:

`</>` Markdown



Nunca mostrar, copiar, mover ni modificar credenciales. Si encuentras credenciales en arc

No es solo seguridad: también reduce el riesgo de que el agente toque zonas sensibles o genere diffs enormes alrededor de credenciales.

Qué quitaría de tu Project Knowledge actual

Quita como contexto permanente:

```
repo completo
CONTEXT.md entero
README.md entero si lo tienes siempre cargado
logs
backups
HTMLs antiguos
docs históricas de Railway
```



Dejaría solo:

```
AI_BRIEF.md
CLAUDE.md
AI_WORKFLOWS.md
AI_NO_READ_BY_DEFAULT.md
```



Y cuando una tarea lo necesite, entonces sí:

```
"Consulta la sección de GitHub Actions de CONTEXT.md"
"Consulta el build pipeline del HTML"
"Consulta la parte de diseño en curso del clasificador"
```



Orden de montaje recomendado

Hazlo en este orden:

1. Crear `CLAUDE.md`.

2. Crear `CLAUDE.local.md` y meterlo en `.gitignore`.
3. Crear `05_Docs/AI_BRIEF.md`.
4. Crear `05_Docs/AI_WORKFLOWS.md`.
5. Crear `05_Docs/AI_NO_READ_BY_DEFAULT.md`.
6. Quitar el repo completo y docs largas de Claude Desktop Project Knowledge.
7. Empezar a usar Claude Code/Aider desde la raíz del repo.
8. Mantener GitHub Desktop para revisión y commits.
9. Rotar/mover secretos.
10. Usar sesiones limpias por tarea.

Resultado esperado

Con esto, cada tarea debería consumir contexto de este estilo:

```
CLAUDE.md
+ AI_BRIEF.md
+ 2-6 archivos relevantes
+ salida de 1-3 comandos
+ diff final
```



en vez de:

```
README completo
+ CONTEXT completo
+ árbol del repo
+ varios archivos abiertos por exploración
+ logs/comandos largos
+ historial de conversación
```



Para este proyecto concreto, el mayor ahorro está en **no usar la documentación larga como memoria permanente** y en obligar al agente a entrar por `rg` + workflows pequeños.